

Ordinare tramite indici

Table of contents

Idea generale	1
Perché è utile	2
Esempio in C: insertion sort sugli indici	3
Esempio in Python: selection sort sugli indici	4

Idea generale

Un algoritmo di ordinamento normalmente modifica direttamente un vettore:

```
v = [30, 10, 20]
```

Dopo l'ordinamento si ottiene:

```
v = [10, 20, 30]
```

In alcuni casi, però, è utile **non spostare i valori del vettore**, ma ordinare un vettore di **indici**.

L'idea è costruire un vettore:

```
idx = [0, 1, 2, ..., n-1]
```

Poi si ordina `idx` confrontando i valori del vettore originale.

Per esempio, dato:

```
v = [30, 10, 20]  
idx = [0, 1, 2]
```

ordinando gli indici in base ai valori di `v`, si ottiene:

```
idx = [1, 2, 0]
```

Infatti:

```
v[1] = 10  
v[2] = 20  
v[0] = 30
```

Quindi il vettore `idx` descrive l'ordine corretto degli elementi, senza modificare direttamente `v`.

Perché è utile

Questa tecnica è utile quando si vogliono ordinare più vettori nello stesso modo, usando un solo vettore come riferimento.

Per esempio:

```
nomi  = ["Luca", "Anna", "Marco"]  
voti  = [24, 30, 27]  
eta   = [21, 20, 22]
```

Se si vuole ordinare tutto in base ai voti, non è necessario ordinare direttamente tutti i vettori. Si può prima calcolare l'ordine degli indici:

```
idx = [0, 2, 1]
```

perché:

```
voti[0] = 24  
voti[2] = 27  
voti[1] = 30
```

Poi si usano gli indici per leggere tutti i vettori nello stesso ordine:

```
nomi[idx[i]], voti[idx[i]], eta[idx[i]]
```

Risultato:

Luca	24	21
Marco	27	22
Anna	30	20

Punto fondamentale per capire l'implementazione.

- il vettore dati `v[]` è l'ingresso della funzione. Viene usato per calcolare gli spostamenti da effettuare, in quanto contiene le chiavi di ordinamento (sorting keys)
- `v[]` NON viene alterato
- il vettore che vien alterato è `idx[]`
- i confronti usano i valori `v[idx[j]]`

Esempio in C: insertion sort sugli indici

In questo esempio non ordiniamo direttamente il vettore `v`, ma il vettore `idx`.

```
#include <stdio.h>

void insertion_sort_indices(int v[], int idx[], int n) {
    for (int i = 1; i < n; i++) {
        int key = idx[i];
        int j = i - 1;

        /* Confronto fatto sui valori v[idx[j]] e v[key] */
        while (j >= 0 && v[idx[j]] > v[key]) {
            idx[j + 1] = idx[j];
            j--;
        }

        idx[j + 1] = key;
    }
}

int main(void) {
    int v[] = {30, 10, 20, 40};
    int n = 4;
    int idx[] = {0, 1, 2, 3};

    insertion_sort_indices(v, idx, n);

    printf("Indici ordinati:\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", idx[i]);
    }

    printf("\nValori letti in ordine:\n");
    for (int i = 0; i < n; i++) {
```

```

        printf("%d ", v[idx[i]]);
    }

    return 0;
}

```

Output:

```

Indici ordinati:
1 2 0 3
Valori letti in ordine:
10 20 30 40

```

Il vettore originale rimane:

```
v = [30, 10, 20, 40]
```

ma l'ordine corretto è descritto da:

```
idx = [1, 2, 0, 3]
```

Esempio in Python: selection sort sugli indici

Anche qui non ordiniamo direttamente `v`, ma il vettore `idx`.

```

def selection_sort_indices(v):
    n = len(v)
    idx = list(range(n))

    for i in range(n - 1):
        min_pos = i

        for j in range(i + 1, n):
            if v[idx[j]] < v[idx[min_pos]]:
                min_pos = j

        idx[i], idx[min_pos] = idx[min_pos], idx[i]

    return idx

voti = [24, 30, 27]
nomi = ["Luca", "Anna", "Marco"]

```

```
eta = [21, 20, 22]

idx = selection_sort_indices(voti)

print("Indici ordinati:", idx)

print("Dati ordinati per voto:")
for i in idx:
    print(nomi[i], voti[i], eta[i])
```

Output:

```
Indici ordinati: [0, 2, 1]
Dati ordinati per voto:
Luca 24 21
Marco 27 22
Anna 30 20
```